

Programming Assignment 1 Report: Aho–Corasick Multi-Pattern Grep

Student: Ethan Fernandez - 25487327

Track: B, Implementation and Building Something

Repo: https://github.com/EJCFernandez/AdvAlgos_A1_AhoCorasick_Grep

Video:

I tried my best to keep this as concise as possible however I kept getting lost in my thoughts and getting distracted by wanting to explain as many things as possible so I apologise for the length. It also doesn't include all/most use as examples but all features are explained in this document.

<https://youtu.be/piLvIamayqc>

a) What I built

I implemented the Aho Corasick multi pattern string matching algorithm (Aho & Corasick, 1975) and built a grep-style command line tool that searches files for many fixed strings in a single pass. It is written in C++17 with CMake and no external dependencies.

The idea behind the algorithm

The interesting part about Aho Corasick is that the algorithm's entire memory of everything it has read so far is one number. That number is a node in a trie and it represents the longest piece at the end of the input that could still be the start of some pattern. When the next byte doesn't continue that piece a failure link tells the algorithm which shorter piece is still alive so it never has to step backwards and re-read the input, therefore every byte is read exactly once no matter how many patterns there are.

A classic example uses the patterns {he, she, his, hers} over the text "ushers" (Aho & Corasick, 1975):

Byte Read	State after the byte	Reported
u	Root (Nothing Matches)	-
s	"s"	-
h	"sh"	-
e	"she"	She and also he via the dictionary link.
r	"she" doesn't have a child to r so it takes its failure link to "he" and then moves to "her"	-
s	"hers"	hers

There are two details that make this work. Firstly a node's failure link is simply its parent's failure link extended by the same byte which only works if the parent's link is already finished. Since every failure link points to a strictly shallower node this lead me to build the links via a breadth-first search which guarantees that the parent's link is already finished. Secondly a pattern can contain other patterns ("he" inside "she") so each node also stores an output link to the nearest shorter pattern that ends at the same place therefore allowing the algorithm to report every match at a byte without walking the whole failure chain again.

Key design decisions

- Alphabet: With the help of AI I decided to use raw bytes (256 symbols) rather than characters this therefore allowed UTF-8 patterns to work as plain byte sequences.
- Data representation: Transitions are stored in a dense table where -1 means no child and node 0 is the root. This costs 1 KiB per node but makes each step a single array lookup therefore trading storage for speed in the final product. The failure links, output links, depths and pattern ids sit in parallel arrays beside it.

- Differences from the textbook version: Instead of converting it into a full Deterministic Finite Automaton (DFA) I kept the classic version that follows failure links at search time and the search is also streaming meaning `search_chunk()` takes a chunk of bytes plus the current state and offset and then returns the new state so the caller can carry it into the next chunk. Furthermore, empty patterns are rejected and duplicate patterns keep separate ids.
- Navigating the code: The algorithm lives in `include/aho_corasick.hpp` (`search_chunk()`) and `src/aho_corasick.cpp` (trie insertion and `build_failure_links()`). The tool itself is split across `src/cli.cpp`, `src/io.cpp` and `src/main.cpp` with `tests/`, `bench/` and `examples/` holding the testers, benchmarks and sample inputs. I wrote `build_failure_links()` and `search_chunk()` myself, while Claude Code wrote everything around them (see Section D).

b) The Tool: ahogrep

What it does

ahogrep takes a set of patterns with two input types either with -e on the command line or one per line from a file with -f and prints every line of the input that contains at least one of them, the same way grep -F does. On top of that it supports -n (line numbers), -c (count matching lines), -o (print each match), -i (ASCII case-insensitive), --color and --dump-automaton which prints every node with its failure and output links and re-checks both invariants. Exit codes are the same as standard grep with 0 meaning something matched, 1 when nothing did and 2 on an error.

How the algorithm sits inside it

The algorithm itself knows nothing about files or lines, instead src/main.cpp reads the input in 1 MiB chunks and hands each one to search_chunk() together with the current state and the byte offset where the chunk starts. The system reports each match as a pattern id and an end offset and then returns its new state for the next chunk. Since the algorithm's whole memory of the past is one integer a match split across two chunks is found without any extra work which is the idea from Section A.

The tool then needs to turn each match into the line it sits on. It does this by scanning outwards from the match for the nearest newline characters which works for two main reasons. Firstly no pattern can contain a newline and secondly my search loop reports matches in order of their end offset because it reports at each byte as it goes. This therefore means the tool can move forward through the file without ever looking back. I personally thought that this is a nice example of the tool depending on a property of the algorithm that isn't obvious from the textbook description.

Interface decisions

In general I tried to make it work like grep wherever possible so that it would be familiar and only make changes to areas that would be beneficial due to the usage of Aho Corasick. One of the main differences is -o which prints every overlapping match rather than grep's leftmost non-overlapping ones. For example if you were to use the patterns abc and abcabc on the text "abcabcabc" with ahogrep it would print five matches while grep prints two. I chose this because it felt like the main advantage of using Aho Corasick was its ability to find overlapping matches in one pass so not utilising this felt like it would defeat the point of the tool.

Furthermore there were a few other smaller decisions that came from edge cases. Empty patterns are rejected with exit code 2 and a message pointing at the line ("line 2 is an empty pattern") because an empty pattern would match everywhere and would make the root a terminal node which complicates the search loop. Additionally on Windows I found that .txt files are formatted slightly differently to include \r in the newlines therefore the tool uses a binary-mode I/O and strips the \r so pattern files saved with Windows gives the same result as a normal one. Additionally, error messages print a single hint line rather than the full usage text since the full text had a tendency to bury the actual error.

Worked example

The examples/ folder contains a short text file and a pattern file that includes the classic he, she, his and hers. Running ahogrep.exe -n -f examples/patterns.txt examples/sample.txt gives the result:

```
1:she sells sea shells by the sea shore
2:the ushers showed his heirs to their seats
4:nothing to see on this line
5:hishers is not a word but it overlaps nicely
6:abccababc: repeated patterns overlap with themselves
7:café and naïve: UTF-8 characters are matched as plain bytes
9:  indented line with his and hers
10:this final line has no trailing newline: she
```

Line 4 is a good example to point out as it doesn't outright contain any of the pattern words but instead contains "this" which contains "his". Furthermore, line 10 demonstrates the newline complications that were mentioned in the previous sections were resolved as the last line is still found without a trailing newline. Running the same search with -c (count matching lines) gives 8 and I've confirmed that the Windows version of .txt still returns the same patterns. Adding -i (ASCII case-insensitive) makes it so that line 3 ("HE SAID HERS") now becomes a hit and still excludes line 8 ("CAFÉ") which is working as intended because -i only folds ASCII letters. Finally ahogrep --dump-automaton on just he, she, his and hers prints 10 nodes and a 10,240-byte table and shows further information about the trie and processing for example it shows fail(she) = "he" and out(she) = "he" which is what allows "he" to be detected inside "she" as mentioned in Section A.

Beyond the direct examples, test_random compares ahogrep against a simple reference matcher on 200,000 random cases (about 56 million matches) covering -i, high bytes, duplicate patterns and five different ways of splitting each input into chunks. I personally ran these and every case passed on both Windows and Linux. Furthermore, in order to test the legitimacy of the tester I created 10 deliberately broken implementations with the help of Claude and it caught all issues it was presented with. Additionally, test_edge (399 checks) and test_io (154 named checks plus 600 random cases at 8 chunk sizes) were used and covered the rest of the tool.

c) What I Learned

What I initially got wrong

My first idea for computing failure links was to strip characters off the front of each node's string and look every shorter suffix up from the root. Technically this is correct however it's incredibly wasteful and didn't even fit the data structure since nodes don't store their strings. It also made the breadth-first order pointless. It wasn't until I understood that a node's failure link is just its parent's failure link extended by one byte that the reason for breadth-first construction actually made sense to me and in addition to that this also opened me to the utilisation of recurrence in the algorithm which I didn't understand before starting.

I also made lots of mistakes while implementing my two functions, the most informative of which I would say was when I was storing the BFS queue as char instead of int. Node ids wrap around once they pass 127, so the code worked perfectly on every hand trace I did and made complete sense to me in theory, however I found out that it would have failed as soon as the trie grew past 127 nodes. Similarly I also computed the case folded byte for -i but then indexed the table with the original data[i] which tests on ordinary lowercase text would never have caught and is why the random tester includes an -i mode. A few smaller mistakes included computing the output link from the parent's failure link instead of the child's, mixing up bytes and node ids (fail_[c]) and a loop that never popped the queue.

The search loop had its own mistake as my first version followed one failure link on a mismatch and then moved on to the next byte instead of retrying the same byte. As an example this misses "hers" in "ushers" because at the r the algorithm has to fall back from "she" to "he" and then try r again. Consequently this taught me that retrying the same byte until it either moves or reaches the root is crucial, which really helped to solidify my understanding of the practicalities of this algorithm.

What surprised me in the benchmarks

I timed ahogrep against GNU grep -F and ripgrep (rg -F) on 100 MB of text taken from ten Project Gutenberg books (Project Gutenberg, n.d.) (Supplied by Claude), using hyperfine (Peter, 2024) and median times. Below are some of the results:

Patterns	ahogrep	grep -F	rg -F	One grep per pattern
1	130ms	62ms	11ms	72ms
100	574ms	928ms	125ms	4.46s
10,000	1.18s	468ms	306ms	-
100,000	8.41s	694ms	458ms	-

The first surprise that I ran into is that the optimal solution isn't the same as the fastest. Aho Corasick has to read every byte so with one pattern ahogrep managed about 769 MB/s while grep's Boyer Moore style skipping (Boyer & Moore, 1977) reached about 1.6 GB/s and ripgrep's SIMD approach (Gallant, 2016) about 9.4 GB/s. However the multi-pattern

approach did clearly beat running grep once per pattern by 7.8 times on natural text and 11 times on random text at 100 patterns. Additionally, ahogrep beat grep in the middle ranges with ahogrep being 3.3 times faster on random text at 100 patterns. After a little bit of research and some back and forth with Claude I originally assumed this was because grep's skipping approach can't skip further than its shortest pattern, but grep switched to Aho Corasick for large pattern sets in version 2.26 (Free Software Foundation, 2016), so this really surprised me and I honestly still don't really have a confirmed explanation for this.

Furthermore, the theory says scan speed shouldn't depend on the number of patterns, yet ahogrep's throughput fell by about 64 times, from 769 MB/s at 1 pattern to 12 MB/s at 100,000 which really surprised me. Claude Code's explanation was that the transition table outgrows the CPU caches. I can see this being true between 10,000 and 100,000 patterns as the table grows from 45 MiB to 555 MiB but it doesn't explain the steepest drop being between 1 and 10 patterns when the table was only 0.1 MiB which should easily fit in cache. My hypothesis is that match density explains the early drop, since half of each pattern set is words sampled from the text, so more patterns means far more matches to report and more output links to process. I didn't record match counts anywhere so this remains a hypothesis but really surprised me from my understanding of the theory.

Finally I was pretty surprised by the actual consequences of the dense table. At 100,000 patterns the automaton has 567,825 nodes, peak memory is 1,059 MiB compared with 56 MiB for grep and 67 MiB for ripgrep and construction taking 1.04 s. Much of that time is zeroing 555 MiB and my BFS checking all 256 slots of every node (about 145 million checks). In hindsight child lists or grouping bytes into classes would fix most of this and I've found out that real implementations like the aho corasick crate (Gallant, n.d.) put so much effort into memory layout for this specific reasoning. The sheer memory difference for such little results was what really surprised me as even with the fastest possible lookup ahogrep was still over 10 times slower than grep at that size which went against my instinct that precomputing tends to be faster for these kinds of projects.

d) AI Use

Tools used and what for

I used two tools heavily throughout the project: Claude in the chat interface and Claude Code in my repository. By line count most of the repository was written by Claude Code, while the two core functions of the algorithm were written by me.

Claude (chat): I used it for initial understanding and planning, explaining concepts such as failure links and output links and reviewing my attempts at `build_failure_links()` and `search_chunk()` over several rounds specifically ensuring it doesn't give me any code and instead was only used to strengthen my understanding. I also used it to help interpret the benchmark results.

Claude Code: It wrote the CMake setup and project scaffold, trie insertion, all three testers, argument parsing, chunked I/O and line tracking, output buffering and colour, `--dump-automaton`, the README and the benchmark and plotting scripts. My CLAUDE.md file told it not to write my two functions. However to check that its tester actually worked it wrote its own version of both functions in a scratch folder and deleted them before I saw them, which I'm Claude disclosed to me.

Where the AI was wrong, unhelpful or misleading

My first example was from Chat Claude which presented me with a benchmark that measured nothing which confused and misled me. It told me to time every tool with its output sent to `/dev/null` and Claude Code implemented this. However GNU grep has a shortcut for when its output goes to `/dev/null` (Free Software Foundation, 2016) and effectively stops at the first match since nothing will be printed, so grep appeared to scan 100 MB in about 1 ms. Claude Code eventually noticed the impossible number and switched every tool to piped output (`hyperfine --output=pipe`), after which the real grep times were 0.06 to 8 s. This taught me not to outright trust AI's conclusions and to read the data and interpret them myself in parallel with the AI.

Chat Claude and Claude Code made a large deal of a CMake bug and created a "Release by default" guard in `CMakeLists.txt` which actually turned out to never fire with MSVC because CMake sets the build type to Debug for Microsoft's compiler before the guard runs. It found and fixed this when I asked it to verify the build but it also claimed the bug would have made my Linux benchmarks run in Debug. When questioned it admitted this was an inference from Windows testing and it had ticked the Linux checklist item using the output of the already fixed build file which didn't actually answer the question. I verified it myself in WSL with a minimal test project and CMake leaves the build type empty with GCC, meaning the original guard would have worked on Linux. This wasted a lot of my time and showed me that AI can correctly fix a bug but still misstate its consequences and that sometimes a small manual experiment can still sometimes beat AI reasoning.

Claude Code reported that one broken version it expected the tester to catch was "provably identical" to the correct code and proceeded to quote two lines that on their own looked like a jump to root on mismatch bug. When I asked about it, it clarified that the summary had left out where those lines actually sat in the code and with that context the lines and the conclusion were unrelated. This once again wasted a bunch of my time so since I'm not

writing every line myself, I learned that a technically correct summary can still be misleading and that I shouldn't make decisions from quoted code taken out of context.

Smaller issues included scripts committed from Windows without executable permission, a first version of `test_io` that took 78 s instead of about 4 s and chat Claude initially providing advice assuming I was on Linux or macOS.

What I understood vs what I took on trust

Understood fully: my two functions, both invariants (every failure link points to a strictly shallower node and the search state is always the longest suffix of the input that is in the trie), why output links are needed and what breaks without them, the design trade-offs in Section A and all the general theory plus some more pointed and deeper theory for Aho Corasick.

Verified by tests but not read line by line: the chunked I/O and line tracking (`test_io`, plus output identical to `grep` on the examples) and the argument parsing (manual testing). I trust `test_random` mainly because it caught all 10 deliberately broken implementations correctly.

Taken on trust: the internals of the benchmark and plotting scripts, where I only checked that the tools' match counts agreed and the numbers were plausible. Additionally I took on trust Claude Code's explanations that peak memory is about double the table size because vectors copy themselves while growing (This made complete sense to me generally so I didn't feel a need to double check this), that `ahogrep`'s extra 1.5 ms of startup comes from zero-filling two 1 MiB buffers and the C++ runtime and the general explanations of how `grep` works internally.

References:

(Some references were provided and explained by Claude to read and help me understand concepts)

- Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6), 333–340.
<https://doi.org/10.1145/360825.360855>
- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10), 762–772.
<https://doi.org/10.1145/359842.359859>
- Free Software Foundation. (2016, October 2). Noteworthy changes in release 2.26 [Release notes]. In GNU grep NEWS.
<https://git.savannah.gnu.org/cgit/grep.git/tree/NEWS>
- Gallant, A. (n.d.). aho-corasick [Computer software]. GitHub.
<https://github.com/BurntSushi/aho-corasick>
- Gallant, A. (2016, September 23). ripgrep is faster than {grep, ag, git grep, ucg, pt, sift}. Andrew Gallant's Blog. <https://blog.burntsushi.net/ripgrep/>
- Peter, D. (2024). hyperfine (Version 1.19.0) [Computer software]. GitHub.
<https://github.com/sharkdp/hyperfine>
- Project Gutenberg. (n.d.). Project Gutenberg. Retrieved September 30, 2026, from <https://www.gutenberg.org/>